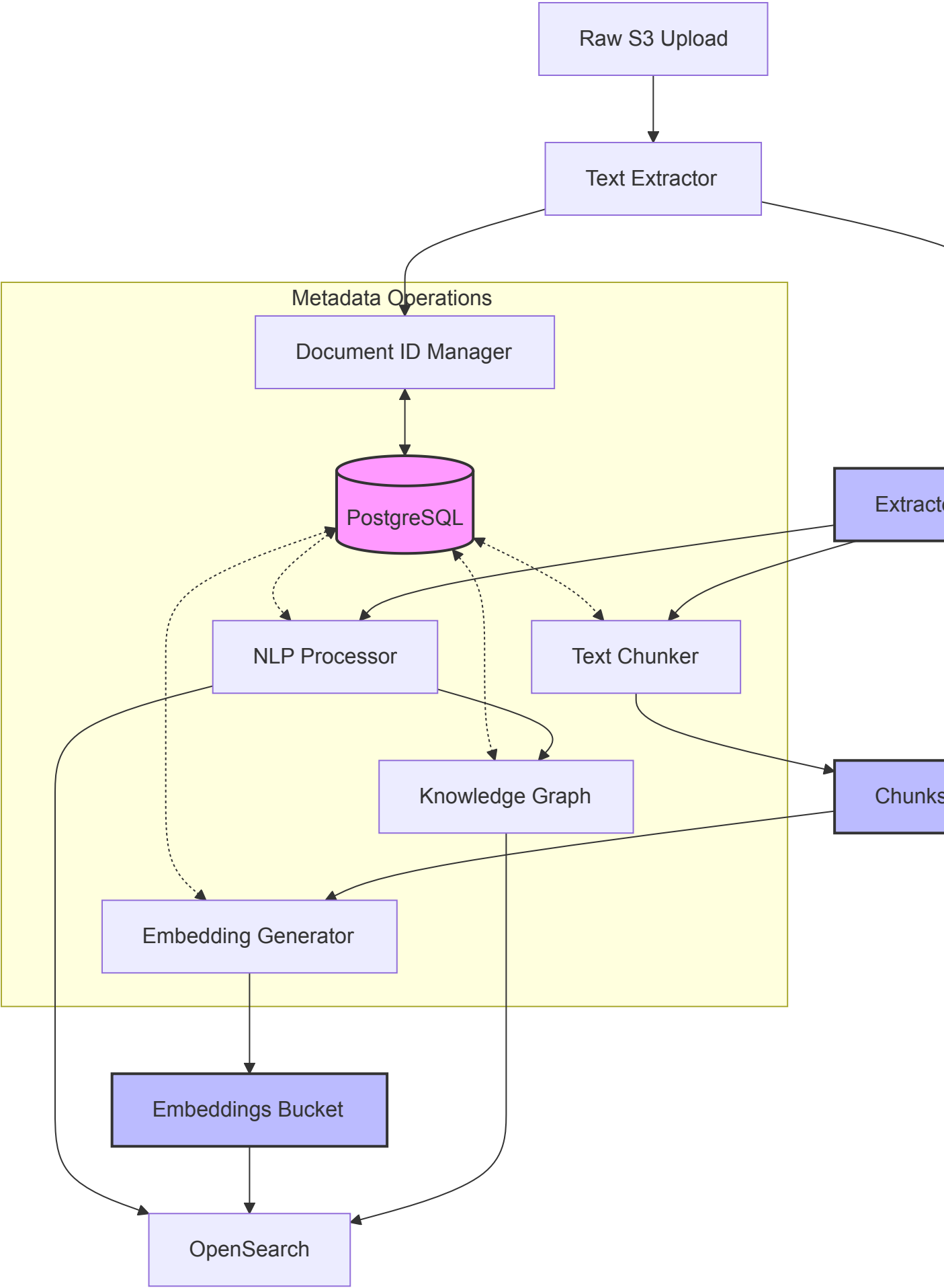


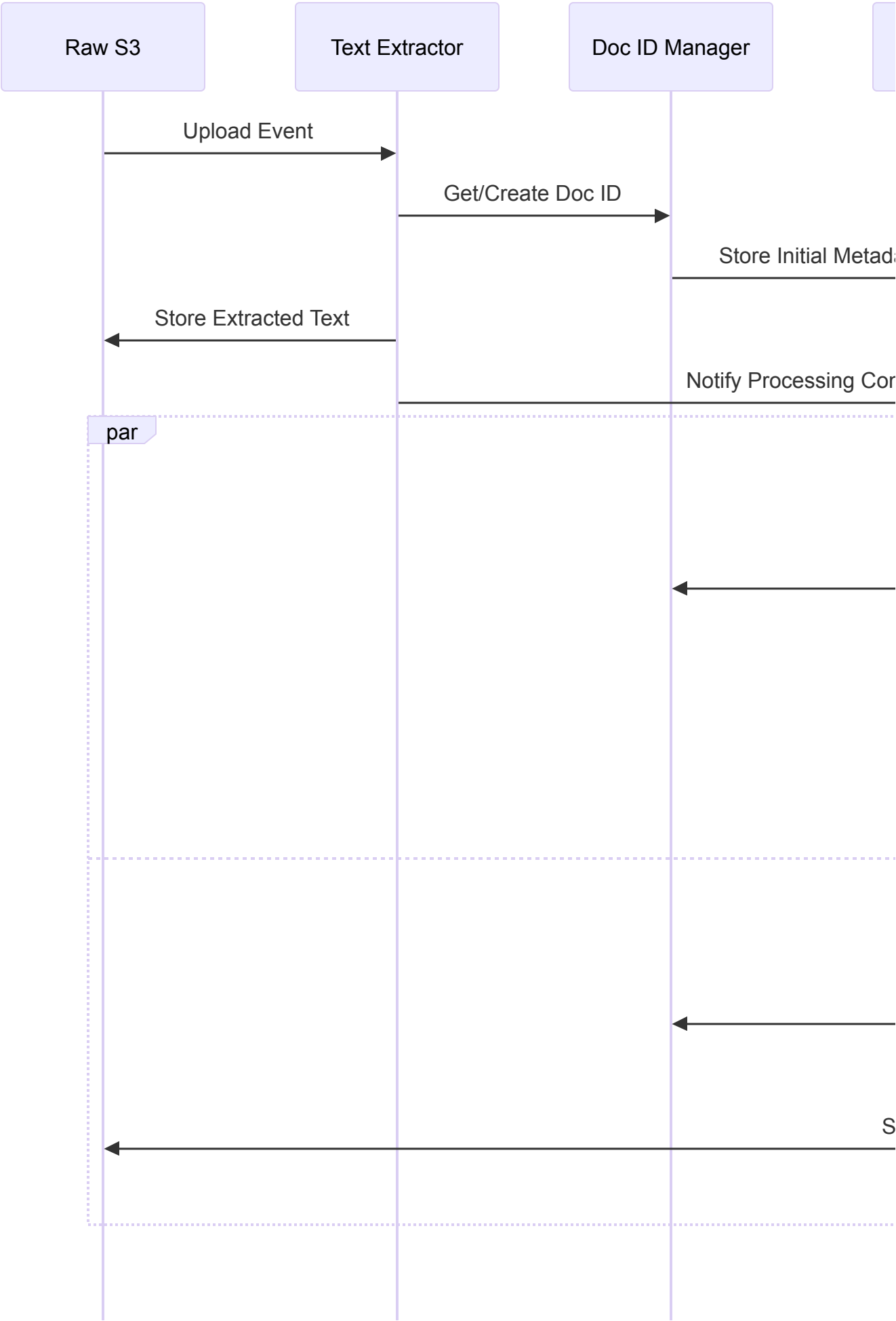
Revised Document Processing Flow

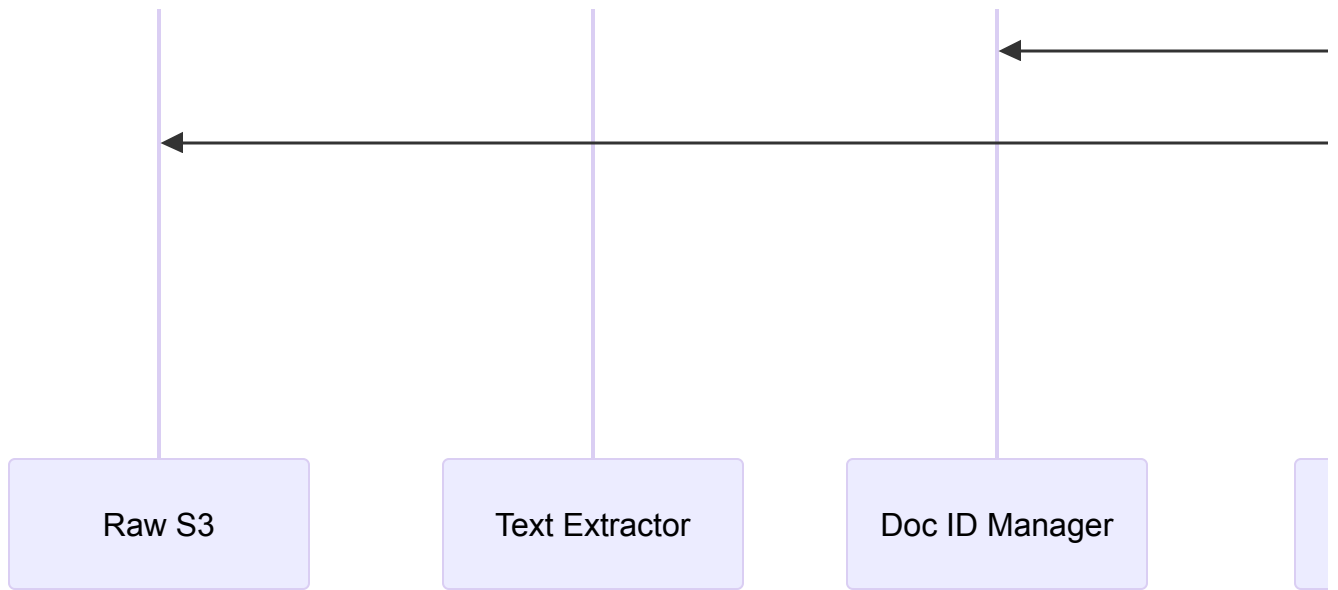
Overview Diagrams

High-Level Processing Flow



Detailed Message Flow





Implementation Details

1. Document ID Manager and Metadata Layer

```
class DocumentIDManager:
    def __init__(self):
        self.db = PostgresConnection()

    def create_document_entry(self, original_name: str, file_hash: str) ->
str:
        """Create new document entry and return document ID"""
        query = """
        INSERT INTO documents (original_name, file_hash, upload_time,
status)
        VALUES (%s, %s, NOW(), 'uploaded')
        RETURNING document_id;
        """
        return self.db.execute(query, (original_name, file_hash))

    def update_processing_status(self, doc_id: str, stage: str, status:
str):
        """Update document processing status"""
        query = """
        INSERT INTO document_processing_status
        (document_id, processing_stage, status, timestamp)
        VALUES (%s, %s, %s, NOW());
        """
        self.db.execute(query, (doc_id, stage, status))
```

2. Text Extractor Lambda

```

class TextExtractor:
    def __init__(self):
        self.s3 = boto3.client('s3')
        self.textextract = boto3.client('textextract')
        self.sns = boto3.client('sns')
        self.doc_manager = DocumentIDManager()

    def process_upload(self, event):
        # Get upload details
        bucket = event['Records'][0]['s3']['bucket']['name']
        key = event['Records'][0]['s3']['object']['key']

        try:
            # Generate document hash and create entry
            file_hash = self.generate_file_hash(bucket, key)
            doc_id = self.doc_manager.create_document_entry(key, file_hash)

            # Extract text using Textract
            text_result = self.extract_text(bucket, key)

            # Store extracted text
            text_key = f"{file_hash}/full_text.json"
            self.s3.put_object(
                Bucket=os.environ['EXTRACTED_TEXT_BUCKET'],
                Key=text_key,
                Body=json.dumps(text_result)
            )

            # Update status and notify
            self.doc_manager.update_processing_status(
                doc_id, 'text_extraction', 'completed'
            )

            # Notify processors
            self.sns.publish(
                TopicArn=os.environ['PROCESSING_TOPIC_ARN'],
                Message=json.dumps({
                    'document_id': doc_id,
                    'file_hash': file_hash,
                    'stage': 'text_extracted'
                })
            )

        except Exception as e:
            self.doc_manager.update_processing_status(
                doc_id, 'text_extraction', f'failed: {str(e)}'
            )

```

```

    )
    raise

def extract_text(self, bucket, key):
    response = self.textextract.detect_document_text(
        Document={'S3Object': {'Bucket': bucket, 'Name': key}}
    )
    return self.process_textextract_response(response)

```

3. NLP Processor Lambda

```

class NLPProcessor:
    def __init__(self):
        self.s3 = boto3.client('s3')
        self.comprehend = boto3.client('comprehend')
        self.doc_manager = DocumentIDManager()
        self.opensearch = OpenSearchClient()

    def process_document(self, event):
        message = json.loads(event['Records'][0]['Sns']['Message'])
        doc_id = message['document_id']
        file_hash = message['file_hash']

        try:
            # Get full text
            text = self.get_full_text(file_hash)

            # Process with NLP
            entities = self.extract_entities(text)

            # Store entities and update KG
            self.store_entities(doc_id, entities)

            # Index full text
            self.index_full_text(doc_id, text)

            # Update status
            self.doc_manager.update_processing_status(
                doc_id, 'nlp_processing', 'completed'
            )

        except Exception as e:
            self.doc_manager.update_processing_status(
                doc_id, 'nlp_processing', f'failed: {str(e)}'
            )

```

```

        raise

    def extract_entities(self, text):
        response = self.comprehend.detect_entities(
            Text=text,
            LanguageCode='en'
        )
        return self.process_entities(response['Entities'])

```

4. Text Chunker Lambda

```

class TextChunker:
    def __init__(self):
        self.s3 = boto3.client('s3')
        self.sns = boto3.client('sns')
        self.doc_manager = DocumentIDManager()

    def process_document(self, event):
        message = json.loads(event['Records'][0]['Sns']['Message'])
        doc_id = message['document_id']
        file_hash = message['file_hash']

        try:
            # Get full text
            text = self.get_full_text(file_hash)

            # Create chunks
            chunks = self.create_structured_chunks(text)

            # Store chunks
            self.store_chunks(doc_id, file_hash, chunks)

            # Update status and notify
            self.doc_manager.update_processing_status(
                doc_id, 'chunking', 'completed'
            )

            self.sns.publish(
                TopicArn=os.environ['CHUNK_TOPIC_ARN'],
                Message=json.dumps({
                    'document_id': doc_id,
                    'file_hash': file_hash,
                    'chunk_count': len(chunks)
                })
            )

```



```

except Exception as e:
    self.doc_manager.update_processing_status(
        doc_id, 'chunking', f'failed: {str(e)}'
    )
    raise

def create_structured_chunks(self, text, chunk_size=5):
    sentences = self.split_into_sentences(text)
    chunks = []

    for i in range(0, len(sentences), chunk_size):
        chunk = sentences[i:i + chunk_size]
        if i + chunk_size < len(sentences):
            # Add overlap
            chunk.extend(sentences[i + chunk_size:i + chunk_size + 2])
        chunks.append(chunk)

    return chunks

```

5. Embedding Generator Lambda

```

class EmbeddingGenerator:
    def __init__(self):
        self.s3 = boto3.client('s3')
        self.bedrock = boto3.client('bedrock-runtime')
        self.doc_manager = DocumentIDManager()
        self.opensearch = OpenSearchClient()

    def process_chunks(self, event):
        message = json.loads(event['Records'][0]['Sns']['Message'])
        doc_id = message['document_id']
        file_hash = message['file_hash']

        try:
            # Get chunks
            chunks = self.get_chunks(file_hash)

            # Generate and store embeddings
            for idx, chunk in enumerate(chunks):
                embedding = self.generate_embedding(chunk)
                self.store_embedding(file_hash, idx, embedding)
                self.index_embedding(doc_id, idx, embedding)

            # Update status

```

```

        self.doc_manager.update_processing_status(
            doc_id, 'embedding', 'completed'
        )

    except Exception as e:
        self.doc_manager.update_processing_status(
            doc_id, 'embedding', f'failed: {str(e)}'
        )
        raise

    def generate_embedding(self, text):
        response = self.bedrock.invoke_model(
            modelId='amazon.titan-embed-text-v1',
            body=json.dumps({'inputText': text})
        )
        return json.loads(response['body'].read())['embedding']

```

6. Entity-Chunk Mapping

```

class EntityChunkMapper:
    def __init__(self):
        self.doc_manager = DocumentIDManager()
        self.kg_client = KnowledgeGraphClient()

    def map_entities_to_chunks(self, doc_id):
        # Get document entities with locations
        entities = self.get_document_entities(doc_id)

        # Get chunk locations
        chunks = self.get_document_chunks(doc_id)

        # Map entities to chunks based on text location
        mappings = []
        for entity in entities:
            matching_chunks = self.find_chunks_containing_entity(
                entity['location'],
                chunks
            )

            if matching_chunks:
                mappings.append({
                    'entity_id': entity['id'],
                    'chunk_ids': matching_chunks,
                    'confidence': entity['confidence']
                })

```

```
# Update Knowledge Graph
self.update_kg_mappings(doc_id, mappings)

return mappings
```

Infrastructure Configuration

1. SNS Topics

```
# In CDK Stack
document_processing_topic = sns.Topic(
    self, "DocumentProcessingTopic",
    display_name="Document Processing Pipeline"
)

chunk_processing_topic = sns.Topic(
    self, "ChunkProcessingTopic",
    display_name="Chunk Processing Pipeline"
)
```

2. SQS Queues

```
# In CDK Stack
nlp_queue = sqs.Queue(
    self, "NLPQueue",
    visibility_timeout=Duration.minutes(30),
    dead_letter_queue={
        "maxReceiveCount": 3,
        "queue": sqs.Queue(self, "NLPLDLQ")
    }
)

embedding_queue = sqs.Queue(
    self, "EmbeddingQueue",
    visibility_timeout=Duration.minutes(15),
    dead_letter_queue={
        "maxReceiveCount": 3,
        "queue": sqs.Queue(self, "EmbeddingDLQ")
    }
)
```

3. Lambda Configurations

```
# In CDK Stack
text_extractor = lambda_.Function(
    self, "TextExtractor",
    runtime=lambda_.Runtime.PYTHON_3_9,
    handler="text_extractor.handler",
    code=lambda_.Code.from_asset("../lambda/text_extractor"),
    timeout=Duration.minutes(15),
    memory_size=1024,
    environment={
        'PROCESSING_TOPIC_ARN': document_processing_topic.topic_arn,
        'EXTRACTED_TEXT_BUCKET': extracted_text_bucket.bucket_name,
        'DB_CONNECTION': db_secret.secret_arn
    }
)
```

Database Schema

```
-- Documents table
CREATE TABLE documents (
    document_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    original_name TEXT NOT NULL,
    file_hash TEXT UNIQUE NOT NULL,
    upload_time TIMESTAMP NOT NULL,
    status TEXT NOT NULL,
    metadata JSONB
);

-- Processing status table
CREATE TABLE document_processing_status (
    status_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    document_id UUID REFERENCES documents(document_id),
    processing_stage TEXT NOT NULL,
    status TEXT NOT NULL,
    timestamp TIMESTAMP NOT NULL,
    details JSONB
);

-- Entity tracking
CREATE TABLE document_entities (
    entity_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    document_id UUID REFERENCES documents(document_id),
    entity_text TEXT NOT NULL,
    entity_type TEXT NOT NULL,
    confidence FLOAT NOT NULL,
    location JSONB NOT NULL,
```

```
        metadata JSONB
    );

-- Chunk tracking
CREATE TABLE document_chunks (
    chunk_id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    document_id UUID REFERENCES documents(document_id),
    chunk_number INTEGER NOT NULL,
    chunk_text TEXT NOT NULL,
    embedding_id UUID,
    metadata JSONB,
    UNIQUE(document_id, chunk_number)
);
```

Would you like me to:

1. Add more implementation details for any component?
2. Expand on the database schema?
3. Add more infrastructure configuration examples?
4. Provide additional error handling examples?